Spring Boot CRUD Application

Library Management — Authors & Books

Name: .....

Register Number: .....

Course / Subject: .....

Date: .....

GitHub URL: .....

Table of Contents

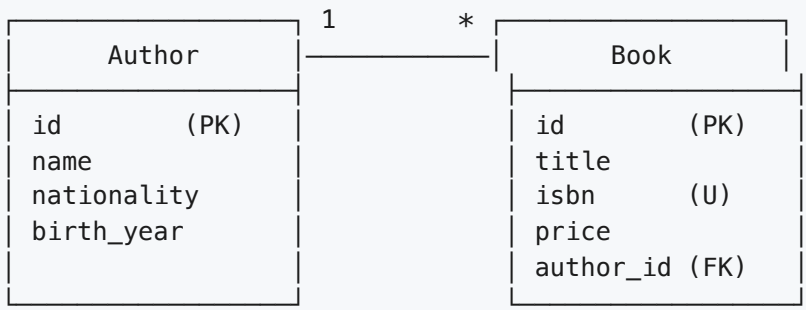
1. Introduction & Tech Stack

This project is a Spring Boot 3 web application that manages two related entities — **Author** and **Book** — using Spring MVC, Spring Data JPA, and an in-memory H2 database. It implements **Create**, **Read**, and **Update** operations through JSP forms, a custom repository **INNER JOIN** query, exception handling for integrity violations, and a suite of unit tests using JUnit 5 and Mockito.

Layer	Technology
Language / Runtime	Java 21
Framework	Spring Boot 3.3.5 (Spring MVC + Spring Data JPA)
ORM	Hibernate (auto-configured)
Database	H2 (in-memory)
View Layer	JSP + JSTL (Jakarta)
Build	Maven 3.9
Testing	JUnit 5, Mockito, AssertJ, Spring Boot Test

2. Entity Relationship Design

The application defines two JPA entities related by a **One-to-Many** association: one `Author` has many `Book`s, and each `Book` belongs to exactly one `Author`. The `Book.isbn` column carries a database-level `UNIQUE` constraint — this is the constraint that demonstrates the integrity-violation exception handling.



Entity	Attribute	JPA / Constraint
Author	id	@Id, @GeneratedValue(IDENTITY)
	name	@Column(nullable=false), @NotBlank
	nationality	@Column(nullable=false), @NotBlank
	birthYear	@Column(name="birth_year", nullable=false), @Min(1000)
Book	title	@Column(nullable=false), @NotBlank

	isbn	@Column(nullable=false, **unique=true**), @NotBlank
	price	@Column(nullable=false, precision=8, scale=2), @Positive
	author	@ManyToOne(optional=false), @JoinColumn(name="author_id")

3. Project Structure

```
spring_project/
├── pom.xml
├── README.md
└── src/
    ├── main/
    │   ├── java/com/example/library/
    │   │   ├── LibraryApplication.java
    │   │   ├── entity/      Author.java, Book.java
    │   │   ├── repository/  AuthorRepository.java, BookRepository.java
    │   │   ├── service/     AuthorService.java,    BookService.java
    │   │   ├── controller/  HomeController, AuthorController, BookController
    │   │   ├── dto/         BookWithAuthor.java
    │   │   └── exception/   GlobalExceptionHandler.java
    │   ├── resources/
    │   │   ├── application.properties
    │   │   ├── data.sql
    │   │   └── static/css/styles.css
    │   └── webapp/WEB-INF/jsp/
    │       ├── common/header.jsp, footer.jsp
    │       ├── books/list.jsp, form.jsp
    │       ├── authors/list.jsp, form.jsp
    │       └── error.jsp
    └── test/java/com/example/library/
        ├── repository/BookRepositoryTest.java
        └── service/   BookServiceTest.java, AuthorServiceTest.java
```

3.1 Maven Dependencies (`pom.xml`)

`pom.xml`

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
</parent>

<properties>
    <java.version>21</java.version>
</properties>

<dependencies>
    <dependency><groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId></dependency>

    <!-- JSP support -->
    <dependency><groupId>org.apache.tomcat.embed</groupId>
        <artifactId>tomcat-embed-jasper</artifactId><scope>provided</scope></dependency>
    <dependency><groupId>jakarta.servlet.jsp.jstl</groupId>
        <artifactId>jakarta.servlet.jsp.jstl-api</artifactId></dependency>
    <dependency><groupId>org.glassfish.web</groupId>
        <artifactId>jakarta.servlet.jsp.jstl</artifactId><version>3.0.1</version>
</dependency>

    <!-- DB and tests -->
    <dependency><groupId>com.h2database</groupId>
        <artifactId>h2</artifactId><scope>runtime</scope></dependency>
    <dependency><groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId><scope>test</scope></dependency>
</dependencies>
```

4. Implementation Details

4.1 Populating the Database

The schema is generated automatically by Hibernate from the JPA entity annotations (`spring.jpa.hibernate.ddl-auto=create-drop`). The file `src/main/resources/data.sql` seeds **10 authors** and **10 books**. The setting `spring.jpa.defer-datasource-initialization=true` ensures `data.sql` runs *after* Hibernate has created the tables.

`src/main/resources/application.properties`

```
# ===== Server =====
server.port=8085

# ===== Datasource (H2 in-memory) =====
spring.datasource.url=jdbc:h2:mem:librarydb;DB_CLOSE_DELAY=-1;MODE=LEGACY
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# ===== H2 Console =====
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# ===== JPA / Hibernate =====
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Run data.sql AFTER Hibernate creates the schema
spring.jpa.defer-datasource-initialization=true
spring.sql.init.mode=always

# ===== JSP view resolver =====
spring.mvc.view.prefix=/WEB-INF/jsp/
spring.mvc.view.suffix=.jsp
```

`src/main/resources/data.sql`

```
-- 10 Authors
INSERT INTO author (name, nationality, birth_year) VALUES ('George Orwell',
'British',   1903);
INSERT INTO author (name, nationality, birth_year) VALUES ('Jane Austen',
'British',   1775);
INSERT INTO author (name, nationality, birth_year) VALUES ('Mark Twain',
'American',  1835);
INSERT INTO author (name, nationality, birth_year) VALUES ('Leo Tolstoy',
'Russian',   1828);
INSERT INTO author (name, nationality, birth_year) VALUES ('Gabriel Garcia Marquez',
'Colombian', 1927);
INSERT INTO author (name, nationality, birth_year) VALUES ('Haruki Murakami',
'Japanese',  1949);
INSERT INTO author (name, nationality, birth_year) VALUES ('Chinua Achebe',
'Nigerian',  1930);
INSERT INTO author (name, nationality, birth_year) VALUES ('Virginia Woolf',
'British',   1882);
INSERT INTO author (name, nationality, birth_year) VALUES ('Ernest Hemingway',
'American',  1899);
INSERT INTO author (name, nationality, birth_year) VALUES ('R. K. Narayan',
'Indian',    1906);

-- 10 Books (some authors have multiple books, some have none -- demonstrates INNER JOIN
behaviour)
INSERT INTO book (title, isbn, price, author_id) VALUES ('1984',
'978-0451524935',  9.99, 1);
INSERT INTO book (title, isbn, price, author_id) VALUES ('Animal Farm',
'978-0451526342',  7.50, 1);
INSERT INTO book (title, isbn, price, author_id) VALUES ('Pride and Prejudice',
'978-0141439518',  6.95, 2);
INSERT INTO book (title, isbn, price, author_id) VALUES ('The Adventures of Tom Sawyer',
'978-0143039563',  8.25, 3);
INSERT INTO book (title, isbn, price, author_id) VALUES ('War and Peace',
'978-0199232765', 14.50, 4);
INSERT INTO book (title, isbn, price, author_id) VALUES ('Anna Karenina',
'978-0143035008', 12.00, 4);
INSERT INTO book (title, isbn, price, author_id) VALUES ('One Hundred Years of Solitude',
'978-0060883287', 11.99, 5);
INSERT INTO book (title, isbn, price, author_id) VALUES ('Norwegian Wood',
'978-0375704024', 10.99, 6);
INSERT INTO book (title, isbn, price, author_id) VALUES ('Things Fall Apart',
'978-0385474542',  9.50, 7);
INSERT INTO book (title, isbn, price, author_id) VALUES ('Malgudi Days',
'978-0140185430',  7.75, 10);
```

OUTPUT 1 — H2 CONSOLE AT `/H2-CONSOLE` AFTER STARTUP

```
SELECT id, name, nationality, birth_year FROM AUTHOR;

ID | NAME                    | NATIONALITY | BIRTH_YEAR
---+-------------------------+-------------+-----------
 1 | George Orwell           | British     |       1903
 2 | Jane Austen             | British     |       1775
 3 | Mark Twain              | American    |       1835
 4 | Leo Tolstoy             | Russian     |       1828
 5 | Gabriel Garcia Marquez  | Colombian   |       1927
 6 | Haruki Murakami         | Japanese    |       1949
 7 | Chinua Achebe           | Nigerian    |       1930
 8 | Virginia Woolf          | British     |       1882
 9 | Ernest Hemingway        | American    |       1899
10 | R. K. Narayan           | Indian      |       1906
(10 rows)


SELECT id, title, isbn, price, author_id FROM BOOK;

ID | TITLE                          | ISBN           | PRICE | AUTHOR_ID
---+--------------------------------+----------------+-------+----------
 1 | 1984                           | 978-0451524935 |  9.99 |     1
 2 | Animal Farm                    | 978-0451526342 |  7.50 |     1
 3 | Pride and Prejudice            | 978-0141439518 |  6.95 |     2
 4 | The Adventures of Tom Sawyer   | 978-0143039563 |  8.25 |     3
 5 | War and Peace                  | 978-0199232765 | 14.50 |     4
 6 | Anna Karenina                  | 978-0143035008 | 12.00 |     4
 7 | One Hundred Years of Solitude  | 978-0060883287 | 11.99 |     5
 8 | Norwegian Wood                 | 978-0375704024 | 10.99 |     6
 9 | Things Fall Apart              | 978-0385474542 |  9.50 |     7
10 | Malgudi Days                   | 978-0140185430 |  7.75 |    10
(10 rows)
```

Both tables are populated with exactly 10 rows each on every startup.

Entity classes

`src/main/java/com/example/library/entity/Author.java`

```
@Entity
@Table(name = "author")
public class Author {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name is required")
    @Size(max = 100)
    @Column(nullable = false, length = 100)
    private String name;

    @NotBlank(message = "Nationality is required")
    @Size(max = 50)
    @Column(nullable = false, length = 50)
    private String nationality;

    @Min(value = 1000, message = "Birth year must be a 4-digit year")
    @Column(name = "birth_year", nullable = false)
    private int birthYear;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Book> books = new ArrayList<>();

    // constructors, getters and setters omitted for brevity
}
```

src/main/java/com/example/library/entity/Book.java

```
@Entity
@Table(name = "book")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Title is required")
    @Size(max = 150)
    @Column(nullable = false, length = 150)
    private String title;

    @NotBlank(message = "ISBN is required")
    @Size(max = 20)
    @Column(nullable = false, unique = true, length = 20)
    private String isbn;

    @NotNull
    @Positive(message = "Price must be greater than 0")
    @Column(nullable = false, precision = 8, scale = 2)
    private BigDecimal price;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "author_id", nullable = false)
    private Author author;

    // constructors, getters and setters omitted for brevity
}
```

4.2 Create Operation

The Add Book form is rendered by `BookController.newForm` and submitted to `BookController.create`. Bean-Validation annotations on the entity (`@NotBlank`, `@Positive`) are checked automatically; field errors render via `<form:errors>`. The same pattern is used for Authors.

BookController.java — create methods

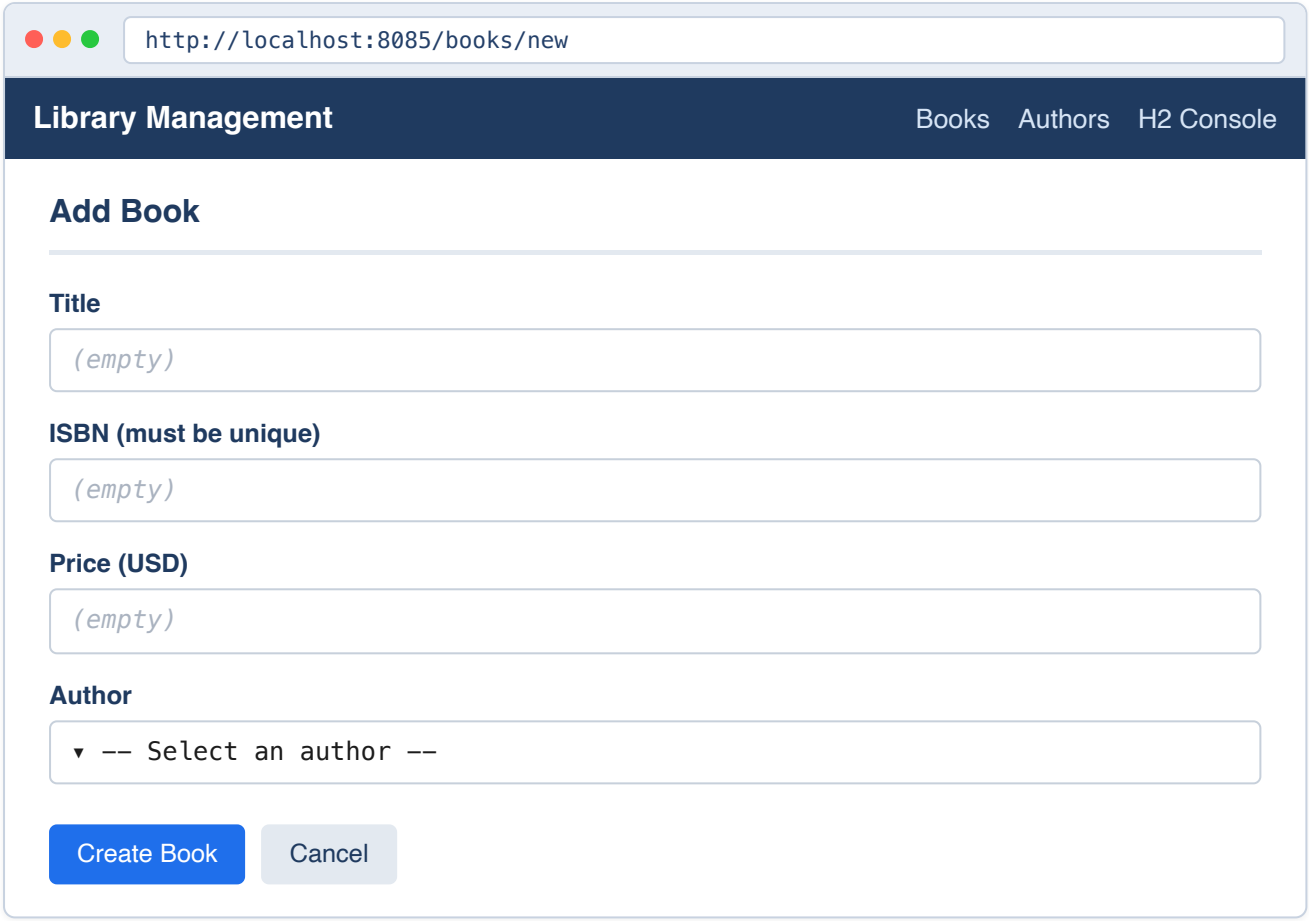
```
@GetMapping("/new")
public String newForm(Model model) {
    model.addAttribute("book", new Book());
    model.addAttribute("authors", authorService.findAll());
    model.addAttribute("mode", "new");
    return "books/form";
}

@PostMapping
public String create(@Valid @ModelAttribute("book") Book book,
                     BindingResult bindingResult,
                     @RequestParam(value = "authorId", required = false) Long authorId,
                     Model model) {
    if (authorId == null) {
        bindingResult.rejectValue("author", "author.required", "Please select an author");
    }
    if (bindingResult.hasErrors()) {
        model.addAttribute("authors", authorService.findAll());
        model.addAttribute("mode", "new");
        return "books/form";
    }
    bookService.create(book, authorId);
    return "redirect:/books";
}
```

BookService.java — create method

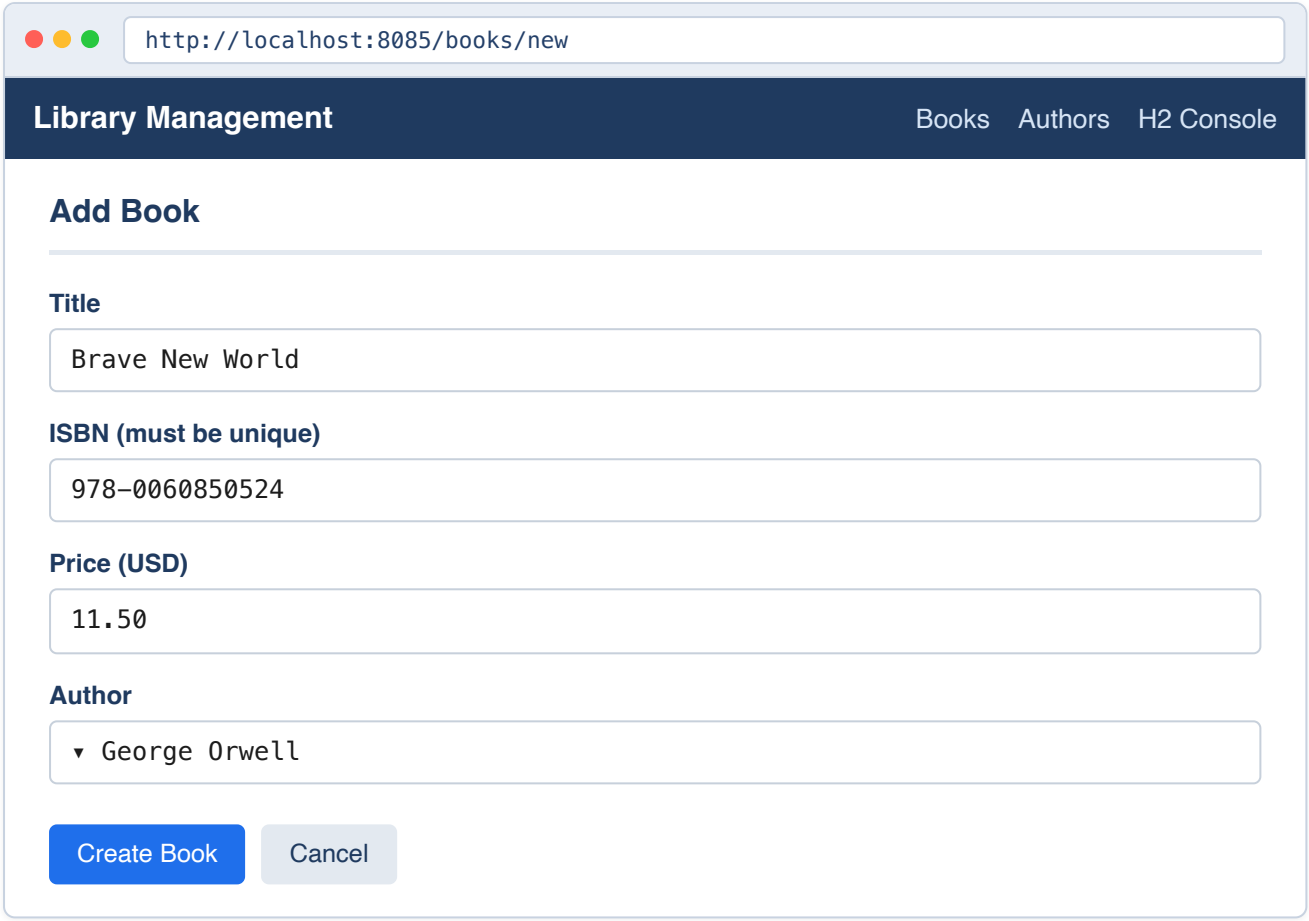
```
public Book create(Book book, Long authorId) {
    Author author = authorRepository.findById(authorId)
            .orElseThrow(() -> new IllegalArgumentException("Author not found: id=" +
authorId));
    book.setAuthor(author);
    return bookRepository.save(book);
}
```

OUTPUT 2 — ADD BOOK FORM RENDERED AT `GET /BOOKS/NEW`



Empty form. Validation errors appear inline below each field.

OUTPUT 3 — SAME FORM FILLED IN WITH A NEW BOOK



A new book ready to be submitted via `POST /books`.

OUTPUT 4 — HTTP TRACE VERIFYING THE ROW IS PERSISTED AND THE LIST GROWS

```
$ curl -i -X POST http://localhost:8085/books \
       -d "title=Brave New World&isbn=978-0060850524&price=11.50&authorId=1"
HTTP/1.1 302
Location: http://localhost:8085/books

$ curl -s http://localhost:8085/books | grep -c '/edit'
11     # the list grew from 10 to 11 rows
```

*Server replies with `302` redirect to `/books`; the list size goes from **10** → **11**.*

4.3 Read Operation — with custom INNER JOIN query

The Books list is produced by a custom JPQL query that performs an **INNER JOIN** between `Book` and `Author` and projects the result directly into the `BookWithAuthor` DTO using JPQL's `SELECT new ...` constructor expression. This avoids the N+1 query problem that would occur with a naive `List<Book>` followed by lazy `book.getAuthor().getName()` calls.

`BookRepository.java`

```
@Repository
public interface BookRepository extends JpaRepository<Book, Long> {

    @Query("""
            SELECT new com.example.library.dto.BookWithAuthor(
                b.id, b.title, b.isbn, b.price, a.name, a.nationality)
            FROM Book b INNER JOIN b.author a
            ORDER BY a.name, b.title
            """)
    List<BookWithAuthor> findAllBooksWithAuthor();
}
```

`BookWithAuthor.java (DTO)`

```
public class BookWithAuthor {
    private final Long bookId;
    private final String title;
    private final String isbn;
    private final BigDecimal price;
    private final String authorName;
    private final String authorNationality;

    public BookWithAuthor(Long bookId, String title, String isbn, BigDecimal price,
                          String authorName, String authorNationality) {
        this.bookId = bookId;
        this.title = title;
        this.isbn = isbn;
        this.price = price;
        this.authorName = authorName;
        this.authorNationality = authorNationality;
    }
    // getters omitted
}
```

Generated SQL (from Hibernate `show_sql`):
`SELECT b1_0.id, b1_0.title, b1_0.isbn, b1_0.price, a1_0.name, a1_0.nationality FROM book b1_0 JOIN author a1_0 ON a1_0.id = b1_0.author_id ORDER BY a1_0.name, b1_0.title` — confirms an INNER JOIN at the SQL level.

OUTPUT 5 — BOOKS LIST AT `GET /BOOKS` (THE INNER-JOIN VIEW)

http://localhost:8085/books

Library Management Books Authors H2 Console

Books

Total: **10** · Produced by INNER JOIN between Book and Author.

#	TITLE	ISBN	PRICE	AUTHOR	NATIONALITY	ACTIONS
1	Things Fall Apart	978-0385474542	$9.50	Chinua Achebe	Nigerian	Edit
2	One Hundred Years of Solitude	978-0060883287	$11.99	Gabriel Garcia Marquez	Colombian	Edit
3	1984	978-0451524935	$9.99	George Orwell	British	Edit
4	Animal Farm	978-0451526342	$7.50	George Orwell	British	Edit
5	Norwegian Wood	978-0375704024	$10.99	Haruki Murakami	Japanese	Edit
6	Pride and Prejudice	978-0141439518	$6.95	Jane Austen	British	Edit
7	Anna Karenina	978-0143035008	$12.00	Leo Tolstoy	Russian	Edit
8	War and Peace	978-0199232765	$14.50	Leo Tolstoy	Russian	Edit
9	The Adventures of Tom Sawyer	978-0143039563	$8.25	Mark Twain	American	Edit
10	Malgudi Days	978-0140185430	$7.75	R. K. Narayan	Indian	Edit

*All 10 books rendered with their author's name and nationality, ordered by author name then title — exactly the JPQL `ORDER BY a.name, b.title`. Note that **Virginia Woolf** and **Ernest Hemingway** have no books seeded and therefore do not appear here — confirming inner-join semantics.*

OUTPUT 6 — AUTHORS LIST AT `GET /AUTHORS`

http://localhost:8085/authors

Library Management Books Authors H2 Console

Authors

#	NAME	NATIONALITY	BIRTH YEAR	# OF BOOKS	ACTIONS
1	George Orwell	British	1903	2	Edit
2	Jane Austen	British	1775	1	Edit
3	Mark Twain	American	1835	1	Edit
4	Leo Tolstoy	Russian	1828	2	Edit
5	Gabriel Garcia Marquez	Colombian	1927	1	Edit
6	Haruki Murakami	Japanese	1949	1	Edit
7	Chinua Achebe	Nigerian	1930	1	Edit
8	Virginia Woolf	British	1882	0	Edit
9	Ernest Hemingway	American	1899	0	Edit
10	R. K. Narayan	Indian	1906	1	Edit

All 10 authors. The # of Books column comes from the `@OneToMany` back-reference on `Author.books`.

4.4 Update Operation

The Edit form is pre-populated with the existing row's values (`BookController.editForm`), and the submission handler (`BookController.update`) loads the existing entity, overwrites the fields, and saves it back. Validation is identical to the create flow.

BookController.java — edit and update methods

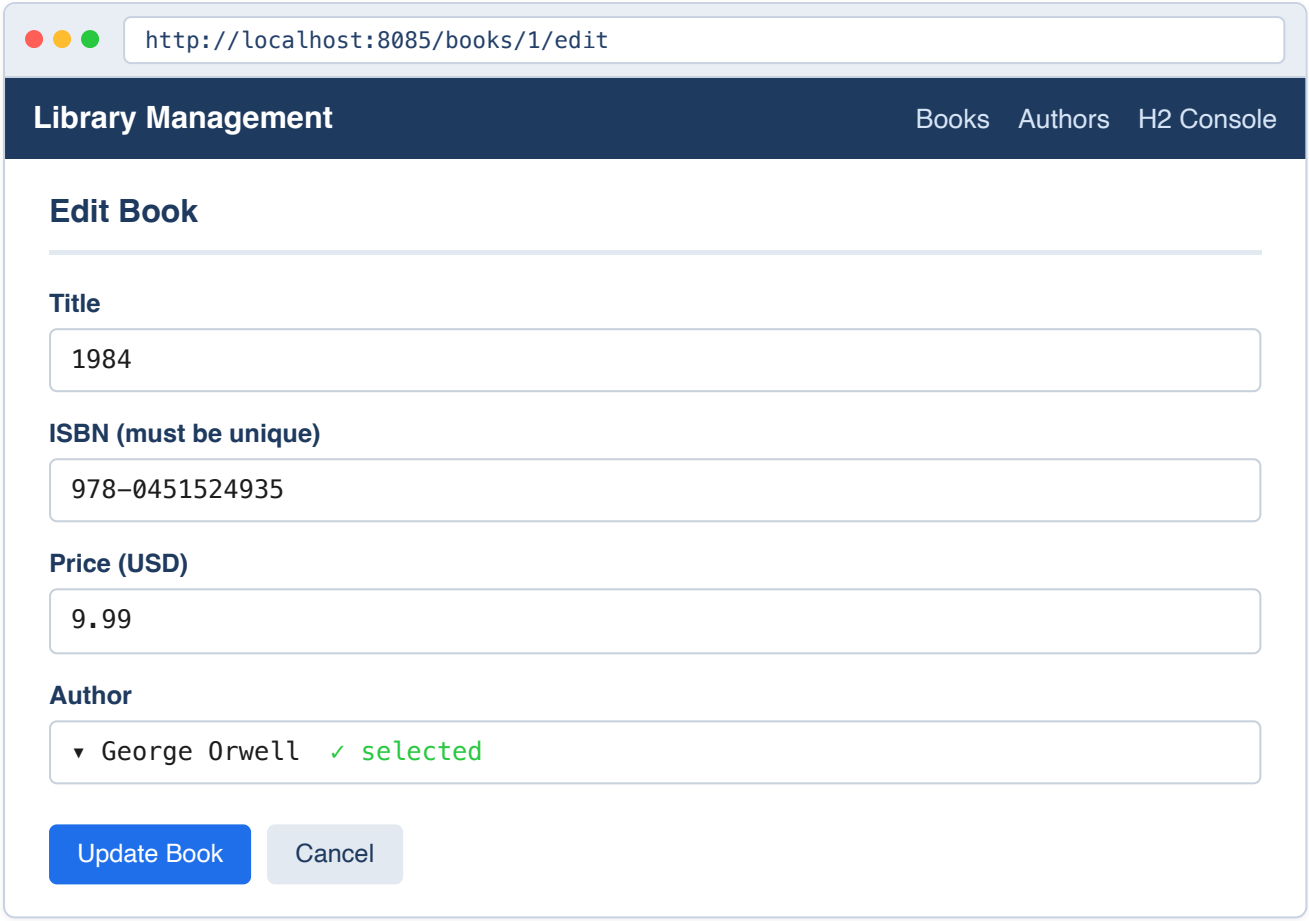
```
@GetMapping("/{id}/edit")
public String editForm(@PathVariable Long id, Model model) {
    Book book = bookService.findById(id);
    model.addAttribute("book", book);
    model.addAttribute("authors", authorService.findAll());
    model.addAttribute("selectedAuthorId", book.getAuthor().getId());
    model.addAttribute("mode", "edit");
    return "books/form";
}

@PostMapping("/{id}")
public String update(@PathVariable Long id,
                     @Valid @ModelAttribute("book") Book book,
                     BindingResult bindingResult,
                     @RequestParam(value = "authorId", required = false) Long authorId,
                     Model model) {
    if (authorId == null) {
        bindingResult.rejectValue("author", "author.required", "Please select an author");
    }
    if (bindingResult.hasErrors()) {
        model.addAttribute("authors", authorService.findAll());
        model.addAttribute("selectedAuthorId", authorId);
        model.addAttribute("mode", "edit");
        return "books/form";
    }
    bookService.update(id, book, authorId);
    return "redirect:/books";
}
```

BookService.java — update method

```
public Book update(Long id, Book updated, Long authorId) {
    Book existing = findById(id);
    Author author = authorRepository.findById(authorId)
            .orElseThrow(() -> new IllegalArgumentException("Author not found: id=" +
authorId));
    existing.setTitle(updated.getTitle());
    existing.setIsbn(updated.getIsbn());
    existing.setPrice(updated.getPrice());
    existing.setAuthor(author);
    return bookRepository.save(existing);
}
```

OUTPUT 7 — EDIT FORM PRE-POPULATED FOR BOOK `ID=1`



The form is filled in from the database row; the correct author is pre-selected in the dropdown.

OUTPUT 8 — BEFORE / AFTER — BOOK `ID=1` UPDATED TO TITLE *"NINETEEN EIGHTY-FOUR"*, PRICE *$12.99*

State	Title	ISBN	Price	Author
Before update	1984	978-0451524935	$9.99	George Orwell
After update	**Nineteen Eighty-Four**	978-0451524935	**$12.99**	George Orwell

```
$ curl -i -X POST http://localhost:8085/books/1 \
       -d "title=Nineteen Eighty-Four&isbn=978-0451524935&price=12.99&authorId=1"
HTTP/1.1 302
Location: http://localhost:8085/books

$ curl -s http://localhost:8085/books | grep -E "Nineteen|12.99"
            <td>Nineteen Eighty-Four</td>
            <td>$12.99</td>
```

Update succeeds (HTTP 302), and the list view confirms the change.

4.5 Exception Handling — Integrity Violations

A `@ControllerAdvice` bean catches `DataIntegrityViolationException` raised by Hibernate (for example when the user tries to insert a book whose ISBN already exists) and renders a clean error page instead of a stack trace.

GlobalExceptionHandler.java

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(DataIntegrityViolationException.class)
    public String handleIntegrityViolation(DataIntegrityViolationException ex, Model model)
{
        String rootMessage = ex.getMostSpecificCause() != null
                ? ex.getMostSpecificCause().getMessage()
                : ex.getMessage();
        model.addAttribute("title", "Data Integrity Violation");
        model.addAttribute("message",
                "The operation could not be completed because it would break a database
constraint "
                        + "(for example: a duplicate ISBN, or a missing author).");
        model.addAttribute("detail", rootMessage);
        return "error";
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public String handleIllegalArgument(IllegalArgumentException ex, Model model) {
        model.addAttribute("title", "Invalid Request");
        model.addAttribute("message", ex.getMessage());
        return "error";
    }

    @ExceptionHandler(Exception.class)
    public String handleGeneric(Exception ex, Model model) {
        model.addAttribute("title", "Unexpected Error");
        model.addAttribute("message", "Something went wrong while processing your
request.");
        model.addAttribute("detail", ex.getMessage());
        return "error";
    }
}
```

OUTPUT 9 — SUBMITTING AN EXISTING ISBN TRIGGERS THE FRIENDLY ERROR PAGE

```
$ curl -s -X POST http://localhost:8085/books \
       -d "title=Duplicate&isbn=978-0451524935&price=5.00&authorId=2"
       | grep -E "Data Integrity|Unique"
    <h2>Data Integrity Violation</h2>
    <p>The operation could not be completed because it would break
       a database constraint (for example: a duplicate ISBN, or a
       missing author).</p>
    <pre>Unique index or primary key violation: "PUBLIC.CONSTRAINT_INDEX_1
       ON PUBLIC.BOOK(ISBN NULLS FIRST) VALUES ( /* 1 */ '978-0451524935' )";</pre>
```

Data Integrity Violation

The operation could not be completed because it would break a database constraint (for example: a duplicate ISBN, or a missing author).

```
Unique index or primary key violation: "PUBLIC.CONSTRAINT_INDEX_1 ON PUBLIC.BOOK(ISBN N
ULLS FIRST) VALUES ( /* 1 */ '978-0451524935' )"
```

Back to Books

Hibernate's `DataIntegrityViolationException` (raised by the unique index on `BOOK.ISBN`) is intercepted by `GlobalExceptionHandler` and rendered as a clean page.

5. Testing — JUnit 5 + Mockito

The project includes **10 unit tests** across 3 test classes, all passing. Repository tests use `@DataJpaTest` with an in-memory H2 instance to verify the inner-join query against a real (test-only) schema. Service tests use Mockito with `@InjectMocks` to isolate the service from the JPA layer.

Test class	Tests	What it verifies
`BookRepositoryTest` (`@DataJpaTest`)	2	The custom INNER JOIN query joins correctly, drops authors with no books, and orders by author name then title.
`BookServiceTest` (Mockito)	5	`create` attaches the right author and persists; `update` overwrites fields; missing ids throw; `findAllJoined` delegates to the repository.
`AuthorServiceTest` (Mockito)	3	Service correctly delegates to the repository, and propagates errors when an author is not found.

`BookRepositoryTest.java`

```
@DataJpaTest
@TestPropertySource(properties = {
        "spring.sql.init.mode=never",
        "spring.jpa.defer-datasource-initialization=false"
})
class BookRepositoryTest {

    @Autowired private TestEntityManager em;
    @Autowired private BookRepository bookRepository;

    @Test
    void findAllBooksWithAuthor_returnsInnerJoinedRows() {
        Author orwell = em.persist(new Author("George Orwell", "British", 1903));
        Author austen = em.persist(new Author("Jane Austen", "British", 1775));
        // Author with NO books -- inner join must drop them
        em.persist(new Author("Author With No Books", "Nowhere", 1900));

        em.persist(new Book("1984",                "ISBN-A", new BigDecimal("9.99"),
orwell));
        em.persist(new Book("Animal Farm",         "ISBN-B", new BigDecimal("7.50"),
orwell));
        em.persist(new Book("Pride and Prejudice", "ISBN-C", new BigDecimal("6.95"),
austen));
        em.flush();

        List<BookWithAuthor> result = bookRepository.findAllBooksWithAuthor();

        assertThat(result).hasSize(3);
        // Ordered by author name then title (G < J alphabetically)
        assertThat(result.get(0).getAuthorName()).isEqualTo("George Orwell");
        assertThat(result.get(0).getTitle()).isEqualTo("1984");
        assertThat(result.get(1).getAuthorName()).isEqualTo("George Orwell");
        assertThat(result.get(1).getTitle()).isEqualTo("Animal Farm");
        assertThat(result.get(2).getAuthorName()).isEqualTo("Jane Austen");
        assertThat(result.get(2).getTitle()).isEqualTo("Pride and Prejudice");

        // Author with no books must NOT appear (inner join semantics)
        assertThat(result).noneMatch(r -> r.getAuthorName().equals("Author With No
Books"));
    }

    @Test
    void findAllBooksWithAuthor_emptyWhenNoBooks() {
        em.persist(new Author("Lonely Author", "Atlantis", 1900));
        em.flush();
        assertThat(bookRepository.findAllBooksWithAuthor()).isEmpty();
    }
}
```

BookServiceTest.java (excerpts)

```
@ExtendWith(MockitoExtension.class)
class BookServiceTest {

    @Mock private BookRepository bookRepository;
    @Mock private AuthorRepository authorRepository;
    @InjectMocks private BookService bookService;

    @Test
    void create_attachesAuthorAndPersists() {
        Book input = new Book("New Title", "ISBN-X", new BigDecimal("12.50"), null);
        when(authorRepository.findById(1L)).thenReturn(Optional.of(author));
        when(bookRepository.save(any(Book.class))).thenAnswer(inv -> inv.getArgument(0));

        Book saved = bookService.create(input, 1L);

        assertThat(saved.getAuthor()).isSameAs(author);
        verify(bookRepository).save(input);
    }

    @Test
    void create_unknownAuthor_throws() {
        when(authorRepository.findById(99L)).thenReturn(Optional.empty());
        assertThatThrownBy(() -> bookService.create(new Book(), 99L))
                .isInstanceOf(IllegalArgumentException.class)
                .hasMessageContaining("Author not found");
        verify(bookRepository, never()).save(any());
    }

    @Test
    void update_missingBook_throws() {
        when(bookRepository.findById(404L)).thenReturn(Optional.empty());
        assertThatThrownBy(() -> bookService.update(404L, new Book(), 1L))
                .isInstanceOf(IllegalArgumentException.class)
                .hasMessageContaining("Book not found");
    }
}
```

OUTPUT 10 — `MVN TEST` OUTPUT (ALL 10 TESTS GREEN)

```
$ mvn test
[INFO] Scanning for projects...
[INFO] ------------------------< com.example:library >-------------------------
[INFO] Building library 1.0.0
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) ---
[INFO] Compiling 12 source files with javac [debug parameters release 21]
[INFO] --- maven-surefire-plugin:3.2.5:test (default-test) ---
[INFO] Running com.example.library.repository.BookRepositoryTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.755 s
[INFO] Running com.example.library.service.BookServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.223 s
[INFO] Running com.example.library.service.AuthorServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.009 s
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  7.541 s
```

All 10 tests across 3 test classes pass; the build succeeds.

6. Challenges Faced

1. JSP with Spring Boot 3 (Jakarta migration)

Spring Boot 3 moved to the **Jakarta EE** namespace. The JSTL imports in JSP files now use `jakarta.tags.core` instead of the older `http://java.sun.com/jsp/jstl/core` URI, and the JSTL implementation artifact had to be `org.glassfish.web:jakarta.servlet.jsp.jstl`. Additionally, JSPs cannot be packaged inside a Spring Boot fat-jar — the app must be launched with `mvn spring-boot:run` so the JSP files in `src/main/webapp/` are picked up by the embedded Tomcat.

2. `data.sql` running before the schema existed

On the first run, the seed inserts failed because Spring tried to execute `data.sql` *before* Hibernate had created the tables. Setting `spring.jpa.defer-datasource-initialization=true` (and `spring.sql.init.mode=always`) re-orders the lifecycle so the schema is created first.

3. Designing the inner-join query without N+1 fetches

A naive approach would be to return `List<Book>` and access `book.getAuthor().getName()` inside the JSP, which would trigger one extra query per book (the N+1 problem). Instead, I used JPQL's `SELECT new com.example.library.dto.BookWithAuthor(...)` constructor expression to project the join straight into a DTO — a single INNER JOIN query that returns everything the view needs.

4. Validation order vs. controller binding

The first version of `Book` had `@NotNull` on the `author` field, which fired during `@Valid` validation *before* the controller had a chance to resolve the `authorId` form parameter to an `Author` entity. The fix was to keep the database-level constraint (`@JoinColumn(nullable=false)`) but remove the `@NotNull` bean-validation annotation — the controller does the manual `rejectValue` when no author is selected.

5. User-friendly errors

Hibernate's raw `DataIntegrityViolationException` message is not user-friendly. A `@ControllerAdvice` translates it into a clean `error.jsp` page with the duplicate-ISBN reason explained, while still surfacing the underlying SQL detail for the developer.

7. How to Run

Prerequisites

- Java 21 (Temurin or any OpenJDK 21 build)
- Maven 3.9+

Commands

```
# Run the application (starts on http://localhost:8085)
mvn spring-boot:run

# Run all unit tests
mvn test

# Build the project
mvn -DskipTests package
```

URLs to verify

URL	Purpose
`http://localhost:8085/`	Redirects to `/books`
`http://localhost:8085/books`	List of all books (uses INNER JOIN query)
`http://localhost:8085/books/new`	Add Book form
`http://localhost:8085/books/1/edit`	Edit Book form (id=1)
`http://localhost:8085/authors`	List of authors
`http://localhost:8085/h2-console`	H2 web console (JDBC URL: `jdbc:h2:mem:librarydb`, user `sa`, no password)

8. GitHub Repository

The complete source code of this project is available at:

(Replace with your GitHub URL after pushing)